

RESUMEN PARA EL EXAMEN — HILOS Y PROCESOS EN JAVA

1 PROCESO

Definición:

Un **proceso** es un **programa en ejecución** que tiene su propia memoria y recursos del sistema operativo.

Cada proceso es **independiente**: si uno falla, no afecta directamente a los demás.

Características:

- Tiene **su propio espacio de memoria**.
- Es gestionado por el **sistema operativo**.
- Se comunica con otros procesos mediante **mecanismos IPC** (pipes, sockets, ficheros...).
- Más **costoso** en recursos que un hilo.

Ejemplo:

```
ProcessBuilder pb = new ProcessBuilder("notepad.exe");  
Process proceso = pb.start();
```

 Lanza un **proceso externo** (el bloc de notas).

2 HILO (THREAD)

Definición:

Un **hilo** (thread) es una **unidad de ejecución dentro de un proceso**.

Los hilos **comparten la memoria** y recursos del mismo proceso.

Características:

- Más **ligero y rápido** que un proceso.
- Comparte memoria → requiere **sincronización**.
- Permite **concurrency** dentro del mismo programa.

3 DIFERENCIAS ENTRE PROCESO E HILO

Característica	Proceso	Hilo
Memoria	Independiente	Compartida entre hilos
Comunicación	Más lenta (IPC)	Más rápida (variables compartidas)
Creación	Costosa	Rápida
Fallo	No afecta a otros procesos	Puede afectar a los demás hilos
Ejemplo	<code>ProcessBuilder</code>	<code>Thread</code> , <code>Runnable</code> , <code>ExecutorService</code>

4 CREACIÓN DE HILOS EN JAVA

♦ Opción 1: Heredar de **Thread**

```
class MiHilo extends Thread {  
    public void run() {  
        System.out.println("Hilo en ejecución");  
    }  
}  
  
new MiHilo().start();
```

♦ Opción 2: Implementar **Runnable**

```
class MiTarea implements Runnable {  
    public void run() {  
        System.out.println("Tarea en ejecución");  
    }  
}
```

```
new Thread(new MiTarea()).start();
```

♦ Opción 3: Usar **ExecutorService**

```
import java.util.concurrent.*;
```

```
ExecutorService executor = Executors.newFixedThreadPool(2);  
executor.submit(() -> System.out.println("Tarea ejecutada por  
pool"));  
executor.shutdown();
```

5 MÉTODOS CLAVE DE **Thread**

Método	Descripción
<code>start()</code>	Inicia el hilo y llama a <code>run()</code> en un nuevo hilo de ejecución.
<code>run()</code>	Contiene el código que ejecuta el hilo (no se llama directamente).
<code>sleep(ms)</code>	Suspende el hilo por un tiempo determinado (milisegundos).
<code>join()</code>	Espera a que otro hilo termine antes de continuar.
<code>interrupt()</code>	Interrumpe un hilo que está esperando o durmiendo.
<code>isAlive()</code>	Devuelve <code>true</code> si el hilo sigue en ejecución.
<code>getName()</code> / <code>setName()</code>	Permite obtener o cambiar el nombre del hilo.

6 ESTADOS DE UN HILO

Estado	Descripción
NEW	Hilo creado, aún no iniciado (<code>new Thread()</code>).
RUNNABLE	Listo o ejecutándose (<code>start()</code>).
BLOCKED	Esperando entrar en una sección sincronizada.

WAITING	Esperando indefinidamente una acción de otro hilo (<code>wait()</code> , <code>join()</code>).
TIMED_WAITING	Esperando un tiempo limitado (<code>sleep()</code>).
TERMINATED	Finalizado (ha terminado <code>run()</code>).

7 CONCURRENCIA Y PARALELISMO

Concepto

Definición

Concurrencia Varios hilos progresan *aparentemente* al mismo tiempo (comparten CPU).

Paralelismo Varios hilos se ejecutan *realmente* a la vez (requiere varios núcleos).

💡 En un procesador multinúcleo puede haber **paralelismo real**; en uno solo, solo **concurrencia simulada**.

8 SINCRONIZACIÓN Y EXCLUSIÓN MUTUA

Definición:

La **sincronización** asegura que los hilos acceden de forma controlada a los **recursos compartidos** (memoria, variables, archivos).

💡 Mecanismos:

- `synchronized` → bloquea acceso a métodos o bloques.
 - `Lock` / `ReentrantLock` → bloqueo manual más flexible.
 - `Semaphore` → controla el número de hilos que pueden acceder.
 - Clases **atómicas** (`AtomicInteger`, etc.) → operaciones seguras sin bloqueos.
-

9 INTERBLOQUEO (DEADLOCK)

Definición:

Ocurre cuando **dos o más hilos se quedan esperando indefinidamente** recursos que el otro posee.

Ejemplo:

```
synchronized (recursoA) {  
    synchronized (recursoB) {  
        // ...  
    }  
}
```

Y otro hilo hace el orden inverso → bloqueo mutuo  

Cómo evitarlo:

- Bloquear recursos **siempre en el mismo orden**.
- Usar `tryLock()` con tiempo de espera.
- Evitar dependencias circulares.



10 API `java.util.concurrent`

♦ ExecutorService

- Gestiona hilos automáticamente.
- Permite ejecutar tareas sin crear hilos manualmente.

```
ExecutorService executor = Executors.newFixedThreadPool(3);  
executor.execute(new MiTarea());  
executor.shutdown();
```

♦ Callable y Future

Permiten **devolver valores** desde hilos.

```
ExecutorService executor = Executors.newSingleThreadExecutor();  
Future<Integer> futuro = executor.submit(() -> 5 + 5);  
System.out.println(futuro.get()); // Espera y muestra 10
```

```
executor.shutdown();
```

11 EJECUCIÓN DE PROCESOS EXTERNOS — **ProcessBuilder**

Definición:

Permite lanzar **programas externos** desde Java.

Ejemplo:

```
try {
    ProcessBuilder pb = new ProcessBuilder("notepad.exe");
    Process proceso = pb.start();
    System.out.println("Bloc de notas iniciado.");

    proceso.waitFor(); // Espera a que se cierre
    System.out.println("Bloc de notas cerrado.");
} catch (Exception e) {
    e.printStackTrace();
}
```

Métodos importantes de **Process**:

Método	Descripción
<code>start()</code>	Inicia el proceso.
<code>waitFor()</code>	Espera a que termine.
<code>destroy()</code>	Termina el proceso.
<code>isAlive()</code>	Comprueba si sigue ejecutándose.
<code>getInputStream()</code>	Lee la salida del proceso.
<code>getErrorStream()</code>	Lee los errores del proceso.

`getOutputStre` Envía datos al proceso.
`am()`

12 CONCEPTOS TEÓRICOS CLAVE

Concepto	Definición breve
Concurrencia	Ejecución de múltiples tareas en un mismo período.
Paralelismo	Ejecución real simultánea en varios núcleos.
Sincronización	Control de acceso a recursos compartidos.
Exclusión mutua	Solo un hilo accede al recurso a la vez.
Deadlock	Bloqueo mutuo entre hilos.
Race condition	Dos hilos modifican el mismo recurso sin control.
Starvation (inanición)	Un hilo nunca obtiene acceso a CPU o recursos.